

Detecting Reentrancy Vulnerabilities for Solidity Smart Contracts With Contract Standards-Based Rules

Jie Cai¹, Jiachi Chen², *Member, IEEE*, Tao Zhang³, *Senior Member, IEEE*,
Xiapu Luo⁴, Xiaobing Sun⁵, and Bin Li

Abstract—The reentrancy vulnerability is one of the most notorious vulnerabilities of smart contracts. It enables attackers to hijack the control flow of a smart contract by invoking a function as the entry point and then re-invoking a function as the reentry point before the execution of the entry point ends. Although several approaches have been proposed to detect this vulnerability, they still face two main limitations. Firstly, existing approaches oversimplify the rules for identifying entry and reentry points, and many even neglect reentry point identification during vulnerability detection. Secondly, most existing approaches overlook the flow of state variables that are not promptly updated, a critical aspect of the reentrancy vulnerability. To address the limitations mentioned above, this article proposes a novel static analysis framework for reentry vulnerability detection. We formulate the reentrancy vulnerability detection as entry and reentry point identification with the state variable flow tracking. Based on the insight that most smart contracts are implemented following various technical standards, we utilize static analysis with standard-based rules to identify potential entry and reentry points. This is achieved by detecting the presence of hijackable and exploitable operations inside the smart contract. Meanwhile, we also conduct state variable flow tracking by the static taint analysis. To verify the effectiveness of our proposed approach, we construct three different datasets. Then We compare our approach with eight state-of-the-art smart contract vulnerability detectors, and our tool outperforms these baselines in detecting more vulnerable samples with fewer false positive samples. Meanwhile, our approach achieves a relatively shorter detection time with better detection results, striking a trade-off between effectiveness and efficiency.

Received 10 September 2024; revised 27 January 2025 and 10 March 2025; accepted 11 March 2025. Date of publication 14 March 2025; date of current version 4 April 2025. This work was supported in part by the Macao Science and Technology Development Fund (FDCT), Macau SAR, under Grant 0161/2023/RIA3; and in part by the National Natural Science Foundation of China, China under Grant 62206238 and Grant 61972335. The associate editor coordinating the review of this article and approving it for publication was Prof. Ghassan Karame. (*Corresponding authors: Tao Zhang; Bin Li.*)

Jie Cai is with the School of Information Engineering, Yangzhou University, Jiangsu 225102, China, and also with the School of Computer Science and Engineering, Macau University of Science and Technology, Macau, China (e-mail: jiecai@yzu.edu.cn).

Jiachi Chen is with the School of Software Engineering, Sun Yat-sen University, Zhuhai 519000, China (e-mail: chenjch86@mail.sysu.edu.cn).

Tao Zhang is with the School of Computer Science and Engineering, Macau University of Science and Technology, Macau, China (e-mail: tazhang@must.edu.mo).

Xiapu Luo is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China (e-mail: csxluo@comp.polyu.edu.hk).

Xiaobing Sun and Bin Li are with the School of Information Engineering, Yangzhou University, Jiangsu 225102, China (e-mail: xbsun@yzu.edu.cn; lb@yzu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3551535

Index Terms—Blockchain, smart contract, reentrancy vulnerability, vulnerability detection.

I. INTRODUCTION

THE smart contract [1] is a program running on the blockchain platform, which can facilitate secure and transparent transactions between users. With the popularization of blockchain technology, smart contract technology has been applied in various domains, including decentralized finance, decentralized exchange, and Non-Fungible tokens, revolutionizing traditional lending, borrowing, and yield farming processes. However, as a relatively new program, smart contracts are susceptible to various vulnerabilities [2]. Among these vulnerabilities, the reentrancy vulnerability is the most notorious, having caused significant financial losses for numerous blockchain users. In the most infamous DAO accident [3], attackers exploited the reentrancy vulnerability inside a smart contract code to steal approximately \$150 million worth of cryptocurrency. Until now, security incidents resulting from the reentrancy vulnerability have continued to occur, seriously impacting user trust in this smart contract technology. Thus, detecting reentrancy vulnerability in smart contracts has become urgent and critical, garnering widespread attention from researchers.

Figure 1 illustrates a generic example of reentrancy vulnerability, where both entry and reentry points correspond to functions within smart contracts. In the entry point, the **Hijackable External Call** refers to a unique operation type that can trigger the hook() function of attackers. Meanwhile, the **Exploitable Operation** inside the reentry point denotes the operations that allow attackers to gain illicit profit. As shown in Figure 1, the main reason for this vulnerability is state variables v_1^s and v_2^s are updated after the hijackable external call, that allows the attacker to reenter the reentry point through its hook() function at steps ② and ③ before updating the value of these state variables at step ⑤. Thus, the outdated value of these state variables can endanger the execution of **Exploitable Operation** within the reentry point following control dependency relations (e.g., v_1^s) or control dependency relations (e.g., v_2^s). Therefore, based on this example, identifying functions with **Hijackable External Call** or **Exploitable Operation** inside as entry or reentry points is fundamental for reentrancy vulnerability detection.

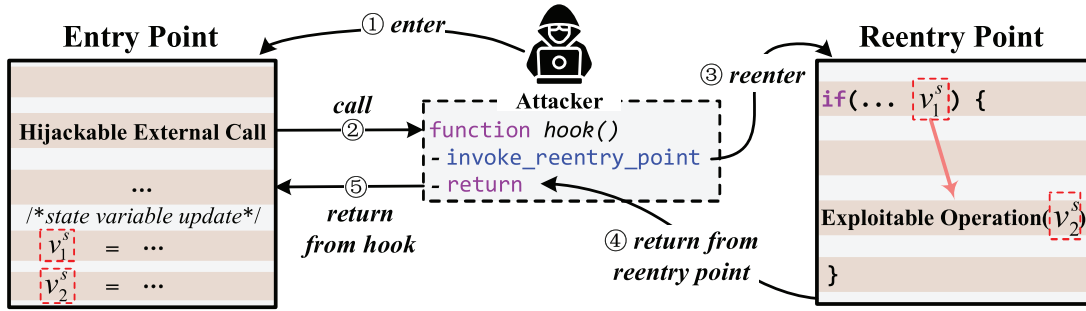


Fig. 1. A generic example of the reentrancy vulnerability.

Recently, several approaches [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [17], [18] have been proposed to detect this vulnerability. The fundamental idea behind these existing approaches is utilizing various technologies, such as symbolic execution or static analysis, to explore the execution space of a smart contract to identify the existence of the code pattern of state variable updating after the hijackable external calls. However, these existing approaches suffer from two limitations during the reentrancy vulnerability detection:

The first limitation is the oversimplified entry and reentry point identification rules used by existing approaches. To identify entry points, as mentioned in Figure 1, the basic idea is to identify the code pattern of state variable updating after the hijackable external calls. However, for the hijackable external call identification, existing approaches often employ either over-broad rules [4], [8], [9] that consider all external calls as hijackable or too-narrow rules that only consider one specific API `.call()` [5], [10], [11] as the hijackable external call. Both of these strategies have their drawbacks and may lead to inaccurate detection results for reentrancy vulnerabilities. Meanwhile, for reentry point identification, most existing approaches neglect this step and solely focus on the simplest scenario, considering the entry point itself as the reentry point. Few approaches [17], [18] utilize the call graph, which can represent the call relationship between functions, to search for additional reentry points. However, reentry points could be any public functions in smart contracts and have no call relations with the entry point. For instance, as shown in Figure 1, the entry and reentry points do not call each other and have no call relations inside the call graph.

The second limitation with existing approaches is to neglect to check whether the state variables, updated after external calls in entry points, can endanger the reentry points. As shown in Figure 1, during re-enter execution (at step ③) by attackers, the outdated value of the state variables v_1^s, v_2^s can affect the exploitable operation through the control or data dependency relations separately inside the reentry point. Thus, the semantics of how the flow of these state variables propagates inside the reentry points are critical for detecting reentrancy vulnerability. However, most existing approaches [4], [5], [7], [11] only focus on identifying the code pattern of state variables updating after the external call inside the smart contract code. The semantics of how the flow of these delayed updated state variables can endanger reentry points is neglected by existing approaches. It limits existing approaches

detecting reentrancy vulnerability when entry and reentry points are two separate functions.

To address the aforementioned limitations, we propose a static analysis based framework for detecting reentrancy vulnerabilities for smart contracts:

For limitation one, we propose to utilize smart contract technical standards with static analysis to identify potential entry and reentry points. Unlike existing approaches that use over-broad or too-narrow rules for entry and reentry points identification, our approach leverages smart contract technical standards. Our key insight is that most smart contracts in the blockchain ecosystem adhere to specific technical standards (e.g., ERC20, ERC721, ERC777). These standards often define several unique operations with unique characteristics, such as triggering hook functions (hijackable external calls) or executing cryptocurrency transfers (exploitable operations), which can cause reentrancy vulnerability if not properly managed. By understanding and utilizing these standards, we can more accurately identify hijackable external calls and exploitable operations during potential entry and reentry points identification. Specifically, during entry point identification, we propose to utilize the technology standard type of the external object to identify whether a specific external call can trigger any hook functions and be a hijackable external call. Similarly, unlike existing approaches that simply consider entry points as reentry points, we propose utilizing technical standards with operation names to identify exploitable operations. The basic idea is that, during re-entered execution by attackers, only the functions containing exploitable operations that allow attackers to gain additional illicit profits can threaten the security of smart contracts. Thus, we propose to utilize static analysis to detect all potentially exploitable operations inside the smart contract. Moreover, all functions containing these exploitable operations are treated as potential reentry points. Meanwhile, the semantic information identified by our approach, including hijackable external calls and exploitable operations, can be helpful in detecting other vulnerabilities. Integrating this semantic information into existing detection approaches, such as Slither, can extend their detection capabilities.

For limitation two, we propose to employ static taint analysis to track the flow of state variables within reentry points. After obtaining potential entry and reentry points, another key step that most existing approaches overlook in reentrancy vulnerability detection is identifying how state

variables, updated after hijackable external calls (e.g., v_1^s, v_2^s in 1), can affect exploitable operations in reentry points. Thus, we propose to utilize static taint analysis to fill this. More specifically, we consider the state variables updated after hijackable external calls as the taint sources, as their value could be outdated during the execution of reentry points. Then, to capture the semantics of how these delayed updated state variables can influence exploitable operations inside reentry points, we utilize taint analysis to track the flow of these state variables in the context of each reentry point to verify whether their flow can propagate to these exploitable operations.

To evaluate our proposed approach, we construct three different datasets. Dataset I contains 23 reentrancy vulnerable samples, which are all exposed and confirmed by the blockchain community and have caused significant losses for blockchain users. Dataset II contains 134 samples with reentrancy vulnerability and 36,366 non-vulnerable samples collected from three existing research [12], [20], [21]. Dataset III contains 22,644 wild smart contracts crawled from the Ethereum blockchain with no ground-truth labels. We have conducted the experiments by comparing our approach with several baselines in Datasets I and II and also evaluate our approach with wild smart contracts in Dataset III. Experimental results demonstrate that our approach can detect all vulnerable smart contracts in Dataset I and 129 vulnerable samples in Dataset II, outperforming all baselines for reentrancy vulnerability detection. Meanwhile, our approach detects that 20.1% of contracts potentially have the reentrancy vulnerability in Dataset III. Furthermore, our approach detects only 531 false positive samples in Dataset II, which is fewer than most baselines.

In summary, the main contributions of this article are as follows:

- We propose a novel static analysis-based framework for reentrancy vulnerability detection, which formulates the reentrancy vulnerability as a combination of entry point identification, reentry point identification, and state variable flow tracking. Furthermore, we propose combining contract standards with taint analysis during reentrancy vulnerability detection.
- We have conducted our experiments by comparing our approach with eight baselines. Experimental results show that our approach is indeed useful in detecting realistic reentrancy vulnerability and outperforms all the baselines.
- We have curated a dataset with confirmed reentrancy-vulnerable smart contracts collected from various forums and existing research, as well as wild contracts directly obtained from the Ethereum blockchain. This dataset is intended to facilitate further research in this field and is available at <https://github.com/smartcontract-detect-yzu/ReDataset>.

II. BACKGROUND AND MOTIVATION

A. Background

1) *State Variable*: The state variable is the type of persistent variable of smart contracts whose value is permanently stored in the blockchain and represents the state of smart contracts. Meanwhile, the state variable is the only way for smart

```

/*entry point*/
1 function borrow (...) public nonReentrant() {
2   if (Exist[msg.sender]==0||Borrowed[msg.sender]==0){
3     /*hijackable external call & exploitable operation*/
4     (IERC721)(token).safeTransferFrom(this,msg.sender,...);
5     Borrowed[msg.sender] = amount;
6   }
7   /*reentry point*/
8   function exit(...) public {
9     if (Borrowed[msg.sender]==0){
10      delete Exist[msg.sender]; //indirect exploitable operation
11    }
12    function repayment(uint amount) public {
13      if (Borrowed[msg.sender] >= amount){
14        Borrowed[msg.sender] -= amount;
15        /*hijackable external call & unexploitable */
16        (IERC721)(token).safeTransferFrom(msg.sender,this,...);
17      }
18    }
19  }

```

Fig. 2. The example of cross-function reentrancy vulnerability with ERC721 token. The entry point is borrow() and reentry point is exit().

contracts to retain and pass the value of their state across various executions.

2) *Internal/External Call*: There are two types of function calls in a smart contract: internal and external calls. The internal call means to call the function inside the current smart contract. Meanwhile, the external call means to call an external contract's function as object.method(). Where the object is an external contract and method() is the function inside the external contract object.

3) *Smart Contract Standard*: The smart contract standard (e.g., ERC20 [19]) refers to a set of specifications that define how smart contracts should be implemented and interact with each other on the blockchain. These standards play a crucial role in facilitating interoperability across different smart contracts, allowing developers to build upon existing solutions and ensuring that contracts can seamlessly interact with each other.

4) *Hook and Fallback Function*: The hook function is a mechanism that allows users of specific smart contracts to implement customized actions automatically in response to predefined actions within these contracts. The standard a smart contract adheres to determines which actions can trigger its users' hook function. For instance, the ERC721 [22] standard requires the ERC721 type smart contract to invoke the hook function onERC721Received() of its users when it conducts the transfer action. Meanwhile, the fallback function is a special hook function in smart contracts that is a default action for smart contracts and does not depend on any standards. If some smart contracts conduct ETH [24] sending, the corresponding receivers' fallback function can be triggered as a default action.

B. Reentrancy Vulnerability

To facilitate easy understanding, we utilize an example in Figure 2 to illustrate the reentrancy vulnerability. This example is a loan smart contract, users can borrow ERC721 tokens [22] through the borrow() function and are prohibited from borrowing again until they have repaid all previously borrowed tokens through the repayment() function.

In this example, the function borrow() acts as the entry point. It is because this function contains the code pattern of the state variable updating (line 4) following the external

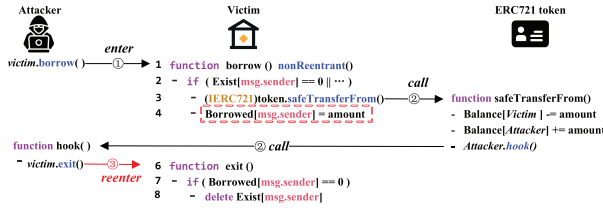


Fig. 3. The process of reentrant vulnerability for the example in Figure 2.

call (line 3) inside. Meanwhile, this external call at line 3 first transfers amount of cryptocurrency asset from current contract this to its caller msg.sender, which can trigger the hook function implemented by msg.sender according to the ERC721 standard [22], making it hijackable for attackers. For the reentry points of this example, both the borrow() and exit() functions are potential candidates. Specifically, the function borrow() can act as a reentry point due to the direct exploitable external call at line 3. As mentioned above, this external call can transfer cryptocurrency assets to anyone who invokes it, allowing attackers to gain illegal assets directly by re-entering this function to execute this operation. We define the operation that allows attackers to obtain cryptocurrency assets directly as a direct exploitable operation. Moreover, for another potential reentry point function exit(), although it does not directly transfer cryptocurrency to attackers, the operation delete Exist[msg.sender] at line 8 allows the attacker to bypass the condition check at line 2 when invoking function borrow() next time. This enables attackers to execute the direct exploitable operation .safeTransferFrom() at line 3 without the need to repay any borrowed assets. Hence, the delete Exist[msg.sender] indirectly enables attackers to gain illegal assets by manipulating the state variable Exist, which is control-dependent on the direct exploitable operation. We refer to this type of operation, which allows for the indirect acquisition of illegal assets, as an indirect exploitable operation. In addition, the function exit() could be a reentry point for attackers to re-enter due to this indirect exploitable operation.

The detailed process of this reentrancy vulnerability is depicted in Figure 3. The attacker first invokes the victim's function borrow() as the entry point. Subsequently, as per the ERC721 standard, the external call (IERC721)(token).safeTransferFrom() at line 3 triggers the attacker's hook function. Leveraging this hook function, the attacker can hijack the control flow before the execution of the state variable updating Borrowed[msg.sender] = amount at line 4 within the borrow() function, and subsequently reenters the exit() function of the victim. In the final step, due to the outdated value of Borrowed[msg.sender], the attacker successfully bypasses the check at line 7 and proceeds to execute the indirect exploitable operation delete Exist[msg.sender] at line 8. This allows the attacker to evade repayment demands and can borrow assets again without fulfilling their repayment obligations. This scenario, wherein the entry and reentry points reside in different functions, is called cross-function reentrancy.

```

1 function withdraw(uint amount) public () {
2   if (WithdrawCnt==0) FirstUser=msg.sender;
3   if (Balance[msg.sender]>=amount){
4     msg.sender.call.value(...);
5     Balance[msg.sender] -= amount;
6   }
7   WithdrawCnt += 1;
8 }

```

Fig. 4. The fixed version of a reentrancy vulnerable function.

C. Motivation

In this section, we introduce the motivations behind our approach.

Observation 1: Tracking the flow of state variables is necessary for reentrancy vulnerability detection. The example shown in Figure 4 presents a fixed version of a reentrancy vulnerability sample, where the order of state variable updating (at line 5) and external call (at line 6) is switched. Despite this fix, several existing tools, including Slither [7], Mythril [9], and Sailfish [8], still incorrectly identify this fixed sample as reentrancy vulnerable. It is caused because these tools just focus on the pattern of state variable updating after external calls and neglect the flow information of these state variables. Specifically, after fixing in Figure 4, there is still a state variable updating WithdrawCnt+=1 at line 8 after the external call .call.value(). Thus, when attackers reenter this function through the hijackable external call before executing the statement WithdrawCnt+=1, the value is WithdrawCnt outdated. However, the outdated value of WithdrawCnt cannot be exploited by attackers to gain cryptocurrency assets as its flow can only propagate to the statement at line 2 and cannot impact the exploitable operation call.value() at line 5. From this case, we can derive the observation that tracking the flow of state variables is necessary for detecting reentrancy vulnerability. The code pattern of state variable updating after an external call can indicate the possibility of certain state variables' values being outdated when attackers reenter potential reentry points. However, without tracking the flow, it is challenging to determine whether these non-updated state variables' values can impact the exploitable operation at the reentry point. Therefore, tracking the flow of state variables becomes crucial in accurately identifying reentrancy vulnerabilities.

Observation 2: The identification of hijackable external calls is critical in detecting entry points for reentrancy vulnerability. For reentrancy vulnerability detection, identifying potential entry points is one basic step. As shown in Figure 2, the main reason for detecting function borrow() as the potential entry point for the reentrancy vulnerability is the existence of state variables updated after an external call (at line 4), which can be utilized to hijack control flow by attackers as shown in Figure 3. Specifically, this external call operation invokes the safeTransferFrom() function, which is implemented inside an external ERC721 Standard type object (IERC721)(token). Although the implementation of the function safeTransferFrom() is unavailable

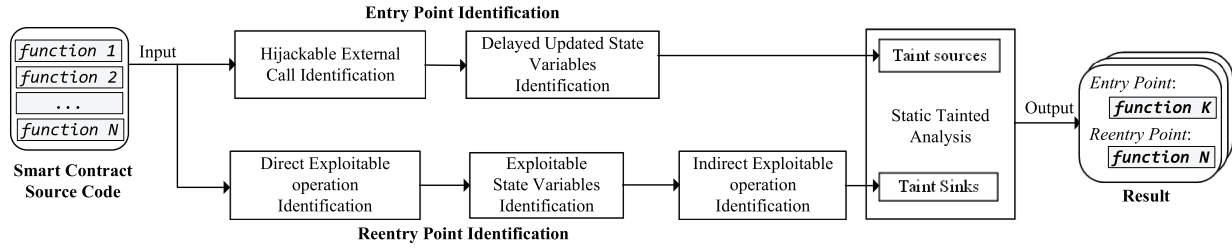


Fig. 5. The framework of our approach.

in the current contract, we can still infer this external call as hijackable based on the calling object type IERC721 at line 3. This is because most smart contracts are implemented following specific technical standards (e.g., ERC721 [22]), and the function `safeTransferFrom()` is hijackable as it triggers the hook function inside the transfer object required by the ERC721 Standard. According to this prior knowledge about ERC721 Standard, the external call at line 3 in Figure 2 should trigger the hook function of the attacker and be hijacked by the attacker to make this function `borrow()` as an entry point for the reentrancy vulnerability. From this case, we can observe that identifying hijackable external calls is crucial for identifying entry points during reentrancy vulnerability detection.

Observation 3: Exploitable operations identification is critical for reentry points detection in reentrancy vulnerability. In reentrancy vulnerability detection, it is vital to identify all potential reentry points within smart contracts. However, it should be noted that not all functions could be reentry points. A reentry point should be a function that contains direct or indirect exploitable operations, enabling attackers to gain illicit profits by re-entering and executing these exploitable operations within the reentry point. Thus, the identification of exploitable operations is necessary to identify all potential reentry points within smart contracts. As shown in Figure 4, the function `withdraw()` is the reentry point as it contains the direct exploitable operation `msg.sender.call.value()` at line 5, which can directly transfer cryptocurrency to anyone who calls it. Similarly, as shown in Figure 2, the function `exit()` should be identified as the reentry point as it contains an indirect exploitable operation at line 8. However, this vulnerable example in Figure 2 is missed by existing tools such as Slither, Mythril, and SoliDetector as they cannot identify this indirect exploitable operation. Moreover, it is important to note that not all transfer operations are exploitable for attackers. For instance, in Figure 2, the function `repayment()` also contains a transfer operation `safeTransferFrom(msg.sender, this, amount)` at line 12. However, this transfer operation does not make the function a reentry point for the reentrancy vulnerability. Meanwhile, the existing tool SailFish [8] incorrectly identifies it as a reentry point. The reason is that the `safeTransferFrom(msg.sender, this, amount)` operation transfers the cryptocurrency from the caller (`msg.sender`) to this contract rather than transferring it out. Thus, when attackers re-enter the function `repayment()` to execute this transfer operation, it will withdraw cryptocurrency from the attackers. This operation is not exploitable for attackers and

should not be considered a reentry point for the reentrancy vulnerability. Thus, from these cases, we can observe that the identification of both direct and indirect exploitable operations is crucial for accurately detecting the reentry points in reentrancy vulnerability detection.

III. APPROACH

Figure 5 provides an overview of our approach. The basic idea behind our approach is that the flow of state variables, which are updated after hijackable external calls in entry points, towards exploitable operations in the reentry point, can effectively reflect the semantics of a reentrancy vulnerability. By tracking this flow, we can effectively detect reentrancy vulnerabilities within smart contracts. Our framework is composed of three components: entry point identification, reentry point identification, and static taint analysis. The framework ultimately identifies and outputs all potential entry and reentry points as detecting results.

A. Entry Point Identification

For entry point identification, our basic idea is to detect the code pattern where state variables are updated after hijackable external calls within each function. First, we identify the hijackable external call in each function. Next, we extract the state variables that are updated following these hijackable external calls. These state variables could act as taint sources for further analysis.

For a smart contract C , we first use the smart contract static analyzer Slither [7] to extract all functions with public or external visibility. This extraction constructs a set F as $F : \{f_1, f_2, \dots, f_n\}$, and $\forall f_i \in F$ can be invoked by users and could act as potential entry or reentry points for attackers. For each function $f_i \in F$, we first utilize Slither [7] to construct its corresponding interprocedural control flow graph (ICFG), denoted as $G_i = (N, E)$, where each node $n \in N$ represents one statement within function f_i and each edge $e \in E$ denotes the control flow between these statements. More specifically, we first utilize Slither to construct the call graph for the contract C . Then, for the target function f_i , we traverse the call graph to obtain the functions called by f_i using a depth-first search. Finally, we utilize Slither again to obtain the CFGs for all the called functions and connect the entry and return nodes between the callee and caller functions to construct the ICFG. Then, we identify the nodes that contain the hijackable external call based on the ICFG G_i . Specifically, in Solidity smart contract, the code for the external call can be written as `object.method()`, where `object` is the external call object

TABLE I

THE COMMON HIJACKABLE EXTERNAL CALL AND THE CALLED OBJECT TYPE IN DIFFERENT STANDARDS

Object Type	Method Name
address [25]	call()
ERC721Receiver [22]	onERC721Received()
ERC721 [22]	safeTransferFrom(), transferFrom()
ERC777 [23]	send(), burn(), operatorSend(), transferFrom()
ERC1155 [26]	safeTransferFrom(), safeBatchTransferFrom()
ERC1155Receiver [26]	onERC1155Received(), onERC1155BatchReceived()

and method() is the called function implemented inside the object. The concrete implementation of method() is inside the object and is unavailable to the caller. Nonetheless, leveraging the prior knowledge of smart contract technical standards, we still can infer whether an external call is hijackable. The main insight is most smart contracts in the current blockchain ecology are implemented according to various standards, and the concrete implementation of each function should follow these standards. Therefore, in Table I, we have summarized widely used smart contract standards with their corresponding functions that can trigger hooks of attackers, rendering them hijackable according to the stipulated standards. Based on this insight, we first search G_i for nodes $n_j \in N$ with content as object.method(). Then, to identify which standard this external object belongs to, we extract the declaration statement of object from the contract code to get its corresponding type as $type^j$. If the external objective type $type^j$ and called method method() are in Table I, we consider this external call is hijackable, and the statement n_j contains a hijackable external call. Finally, we employ n_j as the starting point to conduct the depth-first search on G_i , following the direction of each edge, to identify nodes containing state variable assignment operation inside. These identified state variables are updated after the hijackable external call, and their delayed update is the key reason for the reentrancy vulnerability. All these delayed updated state variables in function f_i construct the set as V_A^{DU} .

As shown in Figure 6, we first scan the ICFG of the function A() for the external call operation as Token.transferFrom() in function _C(). Then, we get the type of the external call object Token as ERC777 through its declaration statements. According to the type ERC777, we can infer that this external call objective Token is implemented according to the standard ERC777 [23]. Moreover, as the external called method name is transferFrom, which can trigger the hook function of attackers, we can identify this external call is hijackable. Then, we extract all state variables updated after this Token.transferFrom() from the ICFG following the control flow to construct the delayed updated state variable set $V_A^{DU} = [v_1^s, v_3^s, v_4^s, Balance]$ for the function A(). Meanwhile, this function A() is identified as an entry point for the reentrancy vulnerability that $F^{en} = [function A()]$.

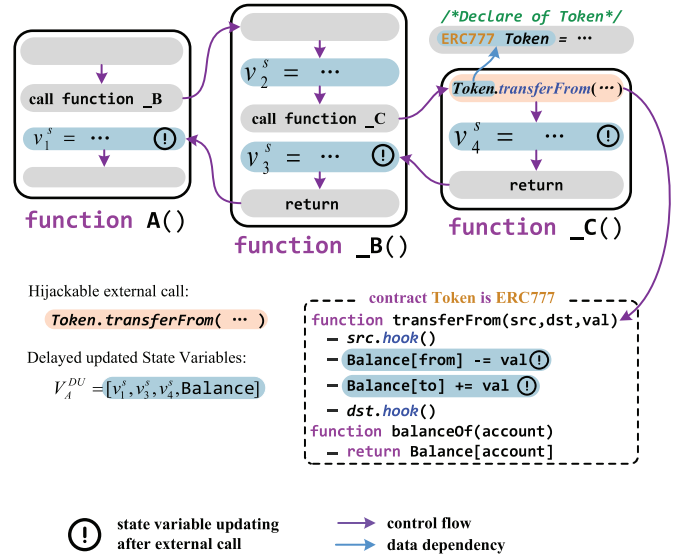


Fig. 6. The process of identifying function A() as the entry point.

TABLE II
TRANSFER METHOD NAMES AND THE PARAMETERS REFER TO THE TRANSFER OBJECTS

Transfer Method Name	Transfer Object	Standard
transfer(to,...)	The 1st param	ERC20
transferFrom(from,to,...)	The 2nd param	ERC20, ERC721, ERC777, ERC1155
safeTransferFrom(from,to,...)	The 2nd param	ERC721, ERC777, ERC1155
mint(to,), _safeMint(to,)	The 1st param	ERC721, ERC777, ERC1155
send(recipient,...)	The 1st param	ERC777
object.call()	The object	All
object.transfer()	The object	All

B. Reentry Point Identification

The reentry point refers to a function allowing attackers to gain illicit financial profits by reentering and executing exploitable operations inside. Thus, to identify reentry points, the basic idea is to **detect the existence of exploitable operations inside each public function**. Moreover, as mentioned in section II-B, exploitable operations can be categorized into direct and indirect types. The direct exploitable operation refers to the operations that directly transfer cryptocurrency to attackers when invoking it. To identify such direct exploitable type operations, we first summarize widely used transfer operations according to various smart contract standards in Table II. Then, we propose conducting data dependency analysis for the object of each transfer operation to determine whether its corresponding transfer objective could be its caller (attacker). Furthermore, indirect exploitable operations refer to operations that can manipulate the state variables direct exploitable operations are dependent on. By manipulating these state variables' value, attackers can indirectly manipulate these direct exploitable operations for illicit profit. Thus, we extract state variables that the direct exploitable operations depended on as exploitable state variables and identify the code fragments that assign these state variables as indirect exploitable operations.

Specifically, for direct exploitable operation identification in each public function $\forall f_i \in F$, we still utilize its corresponding interprocedural control flow graph as $G_i = (N, E)$. Then, we scan all nodes $\forall n \in N$ for statements that conduct transfer operations as listed in Table II, where the first column is the method name of the transfer operation and the second column is the parameter representing the transfer object. Once we identify one statement $n_k \in G_i$ performing the transfer operation, we conduct a backward data dependency analysis for its transfer object. This analysis helps us to determine whether the transfer object depends on the input parameter of the function f_i or the variable `msg.sender`. If the transfer object depends on function input parameters or the `msg.sender`, we can conclude that this transfer operation in statement n_k is directly exploitable within the context of the function f_i . Then, we add the statement n_k to the exploitable operation set N_i^{Exp} of the function f_i . Furthermore, it indicates the function f_i could potentially serve as a reentry point, and we add it to the current contract's reentry point set as $F_c^{re} \leftarrow F_c^{re} \cup f_i$. After getting the direct exploitable operation n_k , we perform the backward data and control dependency analysis to extract all state variables the statement n_k depended on and construct the exploitable state variable set V_k^{Exp} for n_k . Finally, based on V_k^{Exp} , we scan all public functions within the current smart contract for statements that assign these exploitable state variables $v \in V_i^{Exp}$ as indirect exploitable operations. These founded functions are treated as the reentry point and are added to F_c^{re} . Similarly, these statements with indirect exploitable operations are added to their corresponding functions' exploitable operation set N^{Exp} .

C. Static Taint Analysis

After entry and reentry point identification, for the smart contract C , we get its corresponding entry points set F_c^{en} and reentry points set F_c^{re} . Meanwhile, for each entry point $f_i^{en} \in F_c^{en}$, we get its corresponding delayed updated state variable set V_i^{UD} , and the exploitable operation set N_j^{Exp} for each reentry point $f_j^{re} \in F_c^{re}$. In this component, our approach focuses on tracking the flow of these delayed updated state variables to check whether they can endanger exploitable operations within reentry points. The delayed updated state variables are treated as taint sources, while the exploitable operations in reentry points serve as taint sinks. To facilitate this flow tracking, we propose utilizing static tainted analysis to track the propagation of taint from source to sink for reentrancy vulnerability detection.

More specifically, for a pair of entry and reentry point (f_i^{en}, f_j^{re}) , the taint sources are the delayed updated state variables $\forall v^{UD} \in V_i^{UD}$ of the entry point f_i^{en} and the taint sinks are exploitable operations in the reentry point f_j^{re} as $n^{Exp} \in N_j^{Exp}$. The taint analysis aims to track the flow of v^{UD} inside the f_j^{re} to determine if these tainted state variables $v^{UD} \in V_i^{UD}$ can affect the sink n^{Exp} during the execution of the reentry point f_j^{re} . In static taint analysis, we consider both explicit and implicit taint to determine if these tainted state variables can influence exploitable operations. The explicit taint propagates following the data dependency

TABLE III
THE RULES OF TAINT PROPAGATION

Operations	Rules
$v1 = v2$	$\text{taint}(v1) = \text{taint}(v2)$
$v1 = v2 \text{ op } v3$	$\text{taint}(v1) = \text{taint}(v2) \cup \text{taint}(v3)$
if c: block1 else: block2	$\text{taint}(\text{block1}) = \text{taint}(c)$ $\text{taint}(\text{block2}) = \text{taint}(c)$
$v1 = \text{foo}(\dots)$	$\text{taint}(v1) = \text{taint}(\text{return of foo}())$
expr1?expr2:expr3	$\text{taint}(\text{expr1}) = \text{taint}(\text{expr2}) \cup \text{taint}(\text{expr3})$

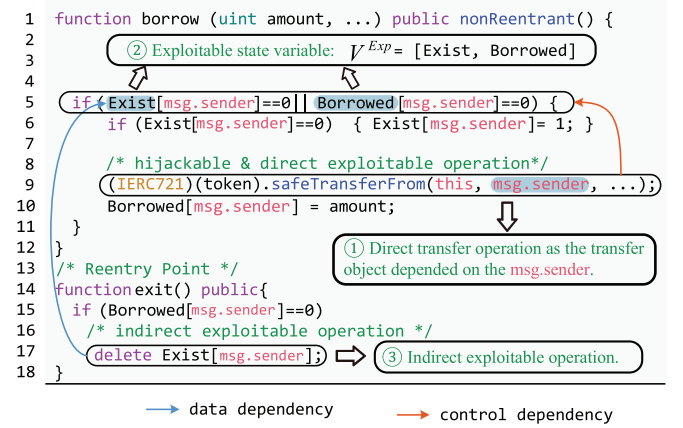


Fig. 7. The inter-contract reentry point identification process of our approach for the example in Figure 2.

relations between statements. Thus, it can reveal how the variables used by exploitable operations are tainted by these delayed updated state variables. Meanwhile, the implicit taint propagates following the control dependency relations between statements that can represent the execution of exploitable operations controlled by these delayed updated state variables. Both the exploitable operations controlled by these delayed state variables or using variables dependent on these delayed state variables can raise the reentrancy vulnerability. Thus, We first utilize Slither to analyze the dominate relations of the condition statements and variable define and use information for each statement to obtain the control and data dependency relations. Then, we utilize the Slither with NetworkX library to traverse these relations by depth-first search as taint propagate. The reachability through the control and data dependency relations determines whether the taint can propagate further, enabling more precise vulnerability detection. Table III shows the taint rules of our approach.

Meanwhile, we also observed two well-known code patterns, nonReentrant and onlyOwner, which are widely used in modern smart contract development to protect against reentrancy vulnerabilities. To account for these protection mechanisms, during taint analysis, we check whether the taint propagation path includes these protection mechanisms. If such protection code is present along the path, we consider the taint propagation to be halted, thereby excluding the path from further consideration for reentrancy vulnerability detection.

Figure 8 shows the taint analysis process for the vulnerable smart contracts in Figures 2. Specifically, the taint source is Borrowed and the sink is an indirect exploitable operation

Taint Source: **[Borrowed]**
Taint Sink: **delete Exist[...]** in function **exit()**

```

1 function exit() public {
2   if (Borrowed [msg.sender] == 0)
3     // Sink: indirect exploitable operation
4     delete Exist [msg.sender];
5 }

```

Fig. 8. The taint propagate for example in Figure 2.

`delete Exist[msg.sender]` at line 4 in the reentry point `exit()`. The taint source can endanger the execution of the sink following the control dependency relations. By following both data and control dependency relations between the reentry points and the sink, we can detect this sample as a reentrancy vulnerability.

IV. EXPERIMENTS

A. Research Questions

RQ1: How effective is our proposed approach in detecting reentrancy vulnerabilities compared to existing approaches? This research question aims to investigate the effectiveness of our proposed approach in detecting reentrancy vulnerabilities compared to existing methods. To address this question, we compare our approach with a set of existing reentrancy vulnerability detection methods, serving as baselines, using two distinct datasets. The first dataset (Dataset I) contains the latest realistic reentrancy vulnerability samples, and the second dataset (Dataset II) contains samples that have already been studied by existing research.

RQ2: How do the individual components of our approach contribute to the overall performance in detecting reentrancy vulnerabilities? This ablation study aims to evaluate the effectiveness of the hijackable external call and exploitable operation identification during entry and reentry point identification, and static taint analysis based entry reentry point pairs identification for reentrancy vulnerability detection.

RQ3: How efficient are our approach and baselines in terms of their time cost for detecting reentrancy vulnerabilities? Efficiency is important for evaluating the performance of reentrancy vulnerability detection approaches. This research question is to investigate whether our approach can make a better trade-off between accuracy and efficiency.

RQ4: Can our approach find reentrancy vulnerabilities in the latest wild smart contracts? This research question intends to evaluate our approach's effectiveness in detecting reentrancy vulnerabilities in practical, real-world smart contracts without ground truth. By applying our approach to a set of real deployed smart contracts obtained from public blockchain platforms, we can assess its ability to detect reentrancy vulnerabilities in a real-world context.

B. Experiment Setup

1) *Dataset*: To answer RQ1 and RQ2, we have constructed two distinct datasets. The dataset I consists of realistic smart contracts that have been exposed as containing reentrancy vulnerabilities, leading to significant financial losses for users.

TABLE IV
THE DATASET I WITH PRACTICAL SMART CONTRACTS OF VARIOUS REENTRANCY VULNERABILITIES

Name	Solc Ver	Loc	Loss
Lendf.Me	0.4.24	2473	\$24,696,616.00
Akropolis	0.5.17	1013	\$2,030,841.00
Rari Capital	0.5.17	3909	\$10,000,000.00
BurgerSwap	0.6.6	337	\$6,800.00
DeFiPie	0.8.15	8448	\$124,999.00
XSURGE	0.8.5	259	\$5,000,000.00
C.R.E.A.M. Finance	0.5.17	2242	\$130,000,000.00
CreatureToadz	0.8.7	593	\$149,600.00
Grim Finance	0.6.12	387	\$30,000,000.00
Visor Finance	0.7.6	661	\$8,200,000.00
HypeBears	0.8.4	873	\$45,730.34
Hundred Finance	0.5.16	1363	\$4,017,100.00
Revest Finance	0.8.7	1699	\$2,000,000.00
BNB Brokers	0.6.10	331	\$3,000,000.00
Fei Protocol	0.8.10	8453	\$10,000,000.00
Thunder Brawl	0.8.7	946	\$702.00
n00dleSwap	0.6.12	670	\$30,000.00
Jaypeggers	0.8.7	616	\$26,044.00
Orion Protocol	0.7.4	2162	\$3,000,000.00
Paribus attack	0.5.17	5038	\$67,800.00
The idols NFT	0.8.9	1445	\$102,000.00
LibertiVault	0.8.17	1634	\$452,000.00
EarningFarm	0.8.3	4180	\$11,000.00
Average	-	2161	\$10,588,646

TABLE V
THE DATASET II COMES FROM DIFFERENT BASELINES

Source	#Count	#loc	#Ver
SmartbugsCurated [20]	31	70.12	0.4.0 ~0.4.25
NFTdefectsRE [12]	62	635.08	0.8.0 ~0.8.15
ReStudy [21]	41	82.71	0.4.0 ~0.6.12
ReStudy Non-Vulnerable [21]	36,366	161.01	0.4.13 ~0.8.7
Total	36,500	-	-

This dataset represents the real-world scenarios of reentrancy vulnerability in the smart contract ecology and enables a comparative analysis between our approach and baselines in detecting reentrancy vulnerabilities within real-world settings. Furthermore, we have curated Dataset II, which comprises reentrancy vulnerable and non-vulnerable samples obtained from existing research studies. This dataset's purpose is to facilitate a comprehensive comparison between our approach and the baselines in previously studied reentrancy vulnerability samples.

More specifically, to construct Dataset I, we investigated multiple forums,¹ for posts that have exposed the vulnerable smart contract projects containing reentrancy vulnerability science 2020. The details of Dataset I are presented in Table IV. To construct Dataset II, we collected smart contracts with reentrancy vulnerability from various sources that researchers had manually verified. The details of Dataset II, including the sources, number of smart contracts, and relevant information, can be found in Table V.

To answer RQ3, we have constructed Dataset III, a large-scale and wild dataset for evaluating whether our proposed approach can find reentrancy vulnerability in wild smart con-

¹All related posts are listed in https://github.com/smartcontract-detect-yzu/ReDataset/blob/main/posts_list

TABLE VI
THE DETECTING RESULT OF ALL BASELINES IN DATASET I

	Ours	Slither	Mythril	Sailfish	Solhint	Securify 1&2	SoliDetector	NFTGuard	Smartian	EF/CF	ItyFuzz
#TP	23	13	3	0	0	0	2	2	0	0	0
#FN	0	10	20	23	23	23	21	21	23	23	23

TABLE VII
THE DETECTING RESULT OF ALL BASELINES IN DATASET II

	Ours		Slither		Mythril		Sailfish		Solhint		Securify1&2		SoliDetector		NFTGuard		Smartian		EF/CF		Ityfuzz	
	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN	#TP	#FN
SmartBugs	31	0	31	0	19	12	21	10	8	23	30	1	28	3	NA	NA	17	14	4	27	0	31
NFTdetectRE	59	3	59	3	1	61	2	60	0	62	0	62	10	52	62	0	0	62	0	62	0	62
ReStudy	39	2	40	1	24	17	24	17	3	38	37	4	34	7	NA	NA	7	34	1	12	1	41
Total	129	5	130	4	44	90	47	87	11	123	67	67	72	62	62	0	24	110	5	129	1	133
	#FP		#FP		#FP		#FP		#FP		#FP		#FP		#FP		#FP		#FP		#FP	
ReStudy Non-Vulnerable	531		2864		8498		952		2622		2077		425		NA		7		14		12	

tracts. More specifically, we first gather the addresses of smart contracts on Ethereum, the most popular blockchain platform, from the sanctuary database [27]. Additionally, we selected samples that users had invoked more than 10 times, ensuring that the smart contracts in this dataset were practically used. In total, we obtained 22,644 smart contracts' addresses. Then, we utilized the Etherscan platform's `getsourcecode()` API to crawl the source code for all these 22,644 smart contracts from the Ethereum blockchain [25] to construct this Dataset III.

Meanwhile, we also analyze the standard categories of these datasets. Dataset I consists of 20 smart contracts adhering to various ERC standards. The distribution is as follows: 5 ERC721 contracts, 2 ERC1155 contracts, 1 ERC777 contract, 2 ERC165 contracts, and 11 ERC20 contracts. Additionally, 2 contracts do not conform to any ERC standard, instead relying on the `native.call()` function for both external calls and exploitable operations. Dataset II contains 62 ERC721 contracts, 4 ERC20 contracts, and 1 ERC1155 contract. The remaining contracts in this dataset are non-standard and also rely on the `native.call()` function for both external calls and exploitable operations. Dataset III includes a total of 19,098 smart contracts, with approximately 84.4% adhering to various ERC standards. The distribution of these contracts is as follows: 22.6% are ERC721 contracts, 2% are ERC1155, ERC777, and ERC165 contracts combined, and 0.1% belong to other standards. The remainder of the contracts are predominantly ERC20.

2) *Baselines*: To answer the first research question, we selected eight state-of-the-art tools that support reentrancy vulnerability detection as the baselines to compare with our proposed approach. More specifically, we first select three industrial-grade tools, i.e., Slither, Mythril, and Solhint. Slither [7] is a static analysis framework that utilizes intermediate representation (SlitherIR) with static analysis for smart contract vulnerability detection. Similarly to our approach, Slither also leverages information based on ERC standards for vulnerability detection, particularly during the identification of entry points. By utilizing well-defined external calls and functions from ERC standards, Slither identifies potential entry points for reentrancy vulnerabilities. Mythril [9] is a concolic analysis-based vulnerability detection tool and serves as a

key component of the commercial smart contract vulnerability detection platform ConsenSys MythX [28]. Solhint [10] is an open-source project to provide a linting utility for smart contracts and has been widely used by several IDEs and companies. Furthermore, we also selected five tools from the top academic conferences or journals as the baselines, including SoliDetector, Sailfish, Securify, NFTGuard, and Smartian. SoliDetector [11] is a knowledge graph-based vulnerability detection approach with SPARQL query [29] for vulnerability detection. Sailfish [8] is a scalable static analysis approach that proposes a storage dependency graph with the value-summary state variable analysis for reentrancy vulnerability detection. Securify [6] is a formal verification vulnerability detection tool that has two versions of code implementations: Securify 1.0 [30] and Securify 2.0 [31]. NFTGuard [12] is the latest vulnerability detection research focusing on detecting vulnerability for smart contracts following the ERC721 Standard by symbolic execution. Similarly to our approach, this baseline also recognizes the potential hijackable external calls defined by the ERC721 standard and leverages them for reentrancy vulnerability detection. The other three baselines, including Smartian [14], EF/CF [16] and ItyFuzz [15] are tools that utilizing fuzzing for vulnerability detection in smart contracts.

V. EXPERIMENTAL RESULTS

A. RQ1: Effectiveness of Our Approach Comparing With Baselines

To answer this research question, we compare our approach against baselines on both Dataset I and Dataset II. We present the true positives and false negatives of each tool. The experimental results are summarized in Table VI and VII. In general, our proposed approach outperforms all baselines for reentrancy vulnerability detection in both Dataset I and Dataset II. Regarding Dataset I, containing the latest realistic reentrancy vulnerable smart contracts, our approach successfully detects all 23 vulnerable samples, whereas the top-performing baseline, Slither, only identifies 12 out of the 23 vulnerable samples. Furthermore, for Dataset II, our approach effectively detects 129 out of the 134 vulnerable samples, with only 531 false positive samples out of 36,366 non-vulnerable samples, surpassing the performance of the baselines.

TABLE VIII
THE TIME-CONSUMING IN SECONDS OF ALL BASELINES IN DATASET I AND II

	Ours	Slither	Mythril	Sailfish	Solhint	Securify 1&2	SolidDetector	NFTGuard
Avg.in Dataset I	13.02	8.97	203.62	246.12	0.61	NA	5.72	NA
Avg. in Dataset II	4.18	0.51	194.78	2.91	0.52	4.41	3.56	469.8
Average	5.38	1.64	202.42	35.86	0.53	4.41	3.85	469.8

More specifically, the baselines Sailfish and Securify failed to analyze most samples in Dataset I and II. The primary reason for these two baselines' poor performance is their inability to properly analyze the majority of the smart contracts in Dataset I due to compatibility issues with the Solidity language. Meanwhile, other baselines with no compatibility issue still failed to achieve satisfactory results of reentrancy vulnerability detection within the practical smart contracts in Dataset I. The first reason is that most baselines still focus on the most primitive reentrancy vulnerability caused by the native hijackable external call interface `.call()` and do not utilize the smart contract standard-related rules for hijackable external call identification. Specifically, for the vulnerable samples collected from NFTRe, most baselines fail to detect them as all samples in the NFTRe dataset are implemented according to the ERC721 standard, which introduces a hook mechanism interface `onERC721Received()` to perform the external call. Thus, most baselines fail to identify the vulnerable samples caused by this external call during vulnerability detection. Instead, our approach utilizes prior domain knowledge about smart contract technical standards to cover various types of smart contracts. The first reason is that some baselines neglect the interprocedural information during vulnerability detection. As shown in 9, the function `buyJay()` contains the hijackable external call at line 6 by calling the function `buyJayWithERC721()`. Meanwhile, following this hijackable external call, this function also conducts the state variable update at line 10 by calling the function `_mint()`. Neglecting the interprocedural information, existing approaches make it difficult to identify external call or state variable update operations that are implemented through function calling. The second reason is most baselines (e.g., SolidDetector or Mythril) focus solely on entry-point identification during reentrancy vulnerability detection. However, since the reentry and entry points may not be the same function, these baselines could overlook the cross-function type reentrancy vulnerability. Meanwhile, the best-performing baseline Slither, which supports the cross-function situation, still misses two vulnerable samples due to neglect of the flow information inside the reentry points. Specifically, the Slither first identifies entry points by identifying the state variables updated after external calls within each function. In Dataset II, the primary cause of reentrancy vulnerabilities in most non-ERC standard samples is the use of the `.call()` API. Our approach can detect these samples by explicitly treating the `.call()` operation as both a hijackable external call and an exploitable operation. Moreover, we observed that fuzzing-based baselines including Smartian, EF/CF, and ItyFuzz did not achieve satisfactory results in detecting reentrancy vulnerabilities. One primary reason is that triggering a reentrancy vulnerability often requires a specific sequence of calls, which

TABLE IX
THE ABLATION STUDY FOR HIJACKABLE EXTERNAL CALL IDENTIFICATION, EXPLOITABLE OPERATION IDENTIFICATION AND TAINT ANALYSIS

Contract Name	External Call	Hijackable External Call	Low Level Call	Direct Exploitable Operation	Indirect Exploitable Operation	Potential Pairs	Tainted Pairs
Lendf.Me	28	8	0	4	13	42	25
Akropolis	175	2	7	12	5	42	7
Rari Capital	67	7	8	5	1	49	7
BurgerSwap	17	4	4	2	3	8	1
DeFiPie	190	13	10	5	2	72	11
XSURGE	20	1	3	2	1	2	1
C.R.E.A.M	60	3	9	4	1	15	3
CreatureToadz	46	3	4	1	2	6	1
Grim Finance	56	7	2	5	0	20	12
Visor Finance	94	1	4	2	0	2	1
HypeBears	54	3	4	2	11	33	4
Hundred Finance	38	3	1	4	3	18	4
Revest Finance	141	5	5	7	3	40	1
BNB Brokers	27	4	4	2	3	6	2
Fei Protocol	190	13	10	5	2	72	11
Thunder Brawl	56	8	5	4	7	48	10
n00dleSwap	25	2	2	1	0	2	1
Jaypeppers	35	9	5	4	3	28	6
Orion Protocol	174	7	7	4	6	45	13
Paribus attack	104	9	3	8	3	108	3
The idols NFT	76	13	4	5	4	126	26
LibertiVault	104	7	11	4	2	24	10
EarningFarm	467	6	7	4	9	54	2
average	98	6	5	4	4	37	7

is difficult for fuzzers to generate through random mutation-based strategies. This limitation significantly increases the time required for fuzzers to explore and construct the correct test sequence. In addition, smart contracts frequently involve complex input parameters, such as 64-byte addresses and other structured data types. This input complexity further constrains the ability of fuzzing-based approaches to generate appropriate inputs, thereby reducing their overall effectiveness in detecting reentrancy vulnerabilities.

Answer to RQ1: By utilizing the flow information with contract standards based rules, our approach can achieve satisfactory results compared with baselines.

B. RQ2: Ablation Study

To answer this research question, we investigate the effectiveness of utilizing smart contract standards-based rules for identifying hijackable external calls and exploitable operations during entry and reentry point identification in Dataset I. Additionally, we assess the effectiveness of identifying potential vulnerable entry-reentry point pairs using static taint analysis. Table IX shows the results in Dataset I.

The results demonstrate that standards-based rules can effectively detect hijackable external calls from all external calls within the samples in Dataset I. Specifically, for hijackable external call identification, which is the first step in

reentrancy vulnerability detection, the use of smart contract standards can exclude an average of 98 external calls as they are identified as non-hijackable. By following these standards, we can effectively exclude non-hijackable external calls from consideration, thereby reducing the number of false positives. Similarly, as exploitable operations are essential features for reentry points, their identification is a critical step in reentrancy vulnerability detection, often overlooked by existing approaches. Utilizing standard-based rules, our approach can effectively identify potential direct exploitable operations within smart contracts, such as money transfers or token minting. Additionally, our approach identifies operations that indirectly affect these direct exploitable operations, enhancing the comprehensiveness of reentry point detection. Specifically, for exploitable operations, our approach utilizes smart contract standards to pinpoint operations that directly involve financial transactions or token management, which are prime targets for reentrancy attacks. We identify an average of 4 direct exploitable operations per contract. By analyzing the flow of operations, we also identify indirect operations that may not immediately seem exploitable but can influence direct exploitable operations, identifying an average of 4 indirect exploitable operations per contract. Based on the results of hijackable external calls and exploitable operation identifications, entry points and reentry points combine to form potentially vulnerable entry-reentry point pairs. According to Table IX, the static taint analysis can effectively identify the pairs where entry points can affect reentry point executions, thereby reducing false positives.

Answer to RQ2: Our approach can effectively identify hijackable external calls and exploitable operations while minimizing entry-reentry point pairs without flow between them, using static taint analysis and contract standards.

C. RQ3: Efficiency of Our Approach Comparing With Baselines

To compare the time consumption of each baseline, we record the duration from smart contract inputting to the detection result obtained. It is worth noting that the time consumption for the dynamic baseline, Smartain, is determined by the preset timeout parameter. Therefore, for this comparison, we focus solely on static analysis-based baselines. The time consumption of all static analysis-based baselines, including Slither, Mythril, Sailfish, Solhint, Securify, SoliDetector, and our proposed approach, on Dataset I and II is presented in Table VIII. According to Table VIII, it can be observed our approaches' time consumption is slightly higher than some baselines but remains within an acceptable range. The main reason is that our approach conducts additional semantic information, such as interprocedural control flow and taint tracking between entry and reentry points. However, most baselines just explore predefined vulnerable code patterns in the plain source code or abstract syntax tree for each smart contract without considering this vulnerability-related semantic information. Additionally, we focus on exploring the

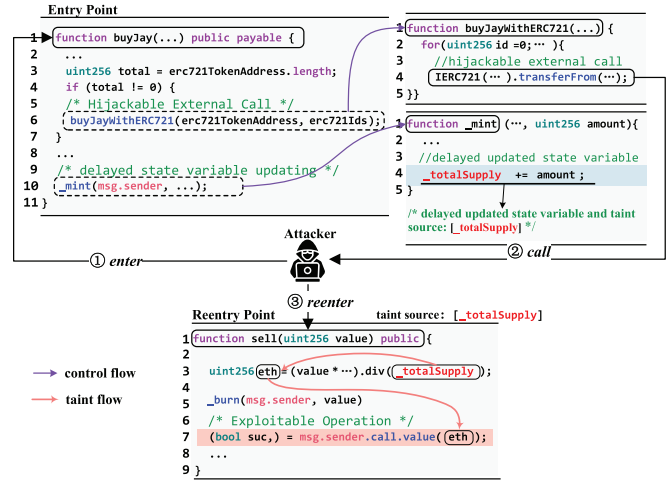


Fig. 9. The cross-function vulnerable sample missed by all baselines with entry point `buyJay()` and reentry point `sell()`.

flow of these taint sources inside the potential reentry points with direct and indirect exploitable operation as taint sinks instead of analyzing all functions as the analyzed targets. By adopting entry and reentry identification, our approach can focus on the critical components that can potentially cause reentrancy vulnerabilities, leading to a good balance between efficiency and effectiveness.

Answer to RQ3: Combining flow information with entry and reentry point identification, our approach achieves a relatively shorter detection time with better detection results, striking a trade-off between effectiveness and efficiency.

D. RQ4: Our Approach on Latest Wild Smart Contracts

To address this research question, we evaluated our approach on Dataset III, consisting of 22,644 wild smart contracts without ground truth, all deployed on the Ethereum blockchain after 2020. The results indicate that our approach identified 20.1% of the smart contracts in Dataset III as potentially containing reentrancy vulnerabilities.

To investigate the effectiveness of our proposed approach, we randomly selected 50 samples labelled as vulnerable by our method and manually checked them for reentrancy vulnerabilities. These selected samples all contained the “state variable update after hijackable external call” code pattern and used these state variables for exploitable operations. Furthermore, we noticed that 31 samples shared similar code patterns, which may cause reentrancy vulnerabilities. The other 19 potentially vulnerable samples are listed in Table X with their entry points, reentry points, and taint sources.

Moreover, for the samples flagged as reentrancy vulnerable by our approach with similar vulnerable code patterns, we manually inspected their code patterns. We notice that the most frequently occurring code pattern identified as potential reentrancy vulnerable by our approach is related to the `mint()` function, which is required by the ERC721 or ERC721A smart contract standards. Within Dataset III, there are about 2,058

TABLE X
PRACTICAL SMART CONTRACT SAMPLES WITH POTENTIAL ENTRY AND REENTRY POINTS IDENTIFIED BY OUR APPROACH

Address	Entry Point	Reentry Point	Taint Source
0x5f43353470Ce421fba6Db6414d631FC050bbABF2	mainMint()	burn()	totalNFT
0xBD9071B63f25Dd199079ED80B3b384D78042956B	veeFriendsMintGrillz()	veeFriendsMintGrillz()	veeFriendsWinners
0xfdf27d7c95eac62d86979e2d73ffbcf380fa9fe	mintCollectibles()	mintCollectibles()	tokenCounter
0x9B46EaE888072fa5f0a6Ab8c5b4ac972Ae548507	redeem()	redeem()	currentIndex
0x97a923ed35351a1382e6bcb5239fc8d93360085	whitelistSummon()	whitelistSummon()	_presaleReserved
0xC7c0a3b4117852F0B4F8ced9a9c447353cAf9d3e	buy()	buy()	nonce
0x22c08c358f62f35b742d023bf2faf67e30e5376e	claim()	mint()	ownerOf
0x6067e1963fe613609ee61e93588e4736cbfc7800	PettaVernissageMint()	StandaPrivateMint()	PettaStandaSupplyMinted
0x4ec83fb4E8EE6955a5B9BD88D901d232205f5444	preSaleWhitelist1()	preSaleWhitelist1()	preSaleMapping
0x9cc103edf0a60e07dd7a6fd4a16fd2cbf7981d92	presalemightygojiras()	presalemightygojiras()	minted
0x96316355c44be69414756d706c61e61aecbd5f3	depositStake()	claimRewards()	IDvsDaysStaked
0x8c47cf966230e75a130f0fe081fdb42633ddcbd7	mintVIP()	controllerBurn()	totalMinted
0x2b56a2e62cc69a4c19c90868abc75751b94d7b36	mintFunkyMonkeyFratHouse()	initDropConfigPirateParade()	drops
0x33328D4149ac498Dd765E8d2eDA920F2D9Bff7f1	publicMint()	burn()	totalNFT
0x24DE7018b2C73B5437eaF647e914a9042CC6D770	mintBio()	withdraw()	_pendingWithdrawals
0x39518fc964559F0537703FB9eb30f23399f50A65	airdrop()	mintToken()	totalReserveMinted
0xdd887ed281e2717d5831189f5d73ef77b8e48379	getRekt()	getRekt()	minted
0xea288f57523a38592a01f580577a5951b62ef906	mintAdmin()	mintAdmin()	nextTokenId
0x85808e40c05Ec5fB440c8893c1018bC78f0a2fcD	reserveTurtles()	mint()	reservedSupply

```

1 function mint(...) {
2   uint256 startTokenId = _currentIndex
3   ...
4   uint256 updatedIndex = startTokenId;
5   do{
6     /* hijackable and exploitable operation */
7     _checkContractOnERC721Received(...updatedIndex++);
8   } while(updatedIndex != end);
9   if(_currentIndex != startTokenId) revert();
10  _currentIndex = updatedIndex;
11 }

```

Delayed updated state variable as the taint source: [_currentIndex]

Answer to RQ4: Our approach still performs well in detecting reentrancy vulnerability for the latest wild and realistic smart contracts.

VI. THREATS TO VALIDATE

In this section, we discuss the threats to the validity of our experimental results.

A. Internal Validity

samples identified as potentially reentrancy vulnerable due to this mint() function. After manual investigation, we noticed that this code pattern indeed contains the reentrancy vulnerability, but most smart contracts also incorporate a code pattern for reentrancy protection when used. When attackers attempt to exploit this reentrancy vulnerability, the reentrancy protection code immediately terminates the current execution and rolls back the state of the current smart contract, effectively preventing any illicit gains by attackers. More specifically, as shown in Figure 10, the state variable `_currentIndex` is updated after the external call and can affect the execution of this exploitable function `mint()`. However, once the attacker reenters this function, the condition checks with `revert()` operation at line 9 can stop the execution and revert the state of the current smart contract. Thus, although this function contains the reentrancy vulnerability, attackers cannot exploit it due to the protection.

Thus, based on the above analysis, as our approach considers more reentrancy-related semantics, it may lead to a higher number of false alarms, particularly for the code with protection mechanisms after the taint paths. However, our approach clearly identifies entry and reentry points along with their corresponding taint sources, enabling smart contract developers to perform further checks. Moreover, because our proposed approach runs relatively quickly, developers can conveniently run analyses multiple times, allowing for rapid validation and refinement of their smart contracts.

Our approach relies on the prior knowledge of smart contract standards to identify hijackable or exploitable operations within the smart contract code. However, as smart contract technology rapidly evolves, our approach may become less suitable for emerging standards. To address this issue, we provide an extension interface to facilitate the inclusion of support for new types of smart contracts as they emerge. Meanwhile, non-ERC standard contracts often implement their own custom external calls and exploitable operations. These operations are closely tied to the application layer and the specific context of the contract. Since our approach relies on static analysis of predefined semantics within the ERC standards, it faces challenges in accurately identifying these custom operations. For the scalability of our approach, we provide the JSON format config file for the user to specify the custom external call and transfer operation. Through this config file, our approach can conduct reentrancy vulnerability detection for the non-ERC standard operations. However, since the delayed-updated state variable identification and taint analysis components of our approach are general and reusable. Smart contract developers can utilize our approach for reentrancy vulnerability detection in non-ERC standard contract by specifying their own external calls and exploitable operations. Additionally, our approach is currently designed for smart contracts implemented in Solidity, which is the dominant programming language for smart contract development. Our proposed approach does not currently support other programming languages used for smart contract development,


```

// The approve() is required by ERC721 to change users' balance
1 function approve(address to, uint tokenId) {
2     Owners[tokenId] = to;
3     Balances[tokenId] += 1;
4 }
5
6 function do_approve(address to, uint tokenId) public {
7     approve(to, tokenId);
8 }

```

Fig. 11. An arbitrary send eth example missed by Slither.

```

1 function indirect_arbitrary_send(address from, address to){
2     uint tokenId = Tokens[from];
3     TokenUser[tokenId] = to;
4 }
5
6 function safe_send(address from){
7     uint tokenId = Tokens[from];
8     address to = TokenUser[tokenId];
9     to.send(Balances[tokenId]);
10 }

```

Fig. 12. An indirect arbitrary send eth example missed by Slither.

such as Vyper [33], or Cairo [34]. Furthermore, our approach heavily relies on various semantic information within the smart contract's source code to aid the analysis. This source-code dependency makes our approach a valuable tool for developers during the smart contract development phase. However, it may not be directly applicable to smart contracts with no source code but only bytecode.

B. External Validity

As a new form of program, there is a lack of a standardized reentrancy vulnerability dataset with ground-truth labels. We have constructed our dataset with ground truth by collecting vulnerable smart contracts from social media and existing research. Then, we evaluated our approach on this dataset of a limited number of samples confirmed by researchers and users with reentrancy vulnerabilities. Although the results of the experiment show the effectiveness of our approach, it is still quite possible that we may miss some corner cases in our dataset. Thus, we also have collected and crawled 22,644 real-world contracts with source code from the Etherscan platform to further investigate the performance of our approach. We share our datasets to aid future research in this area.

VII. RELATED WORK

A. Static Analysis Based Reentrancy Vulnerability Detection

For the static analysis-based reentrancy vulnerability detection, their main idea is to search the execution space of a smart contract for the pre-defined reentrancy vulnerability code patterns. Oyente [4] is the first smart contract vulnerability detection tool that uses predefined vulnerable patterns with symbolic execution for vulnerability detection. It utilizes the bytecode of the smart contract with the SMT solver to conduct symbolic execution to explore all potential execution paths on the control flow graph for vulnerability detection. Smartcheck [5] is a syntax pattern-based vulnerability detection work for smart contracts. It utilizes the abstract syntax trees (AST) with the XML parse trees for vulnerability pattern searching and detection. Securify [6] is a security property analysis framework for the smart contract. It checks the existence of pre-defined security properties through a specialized domain-specific language for vulnerability detection. Slither [7] is an industrial-grade smart contract analysis framework. It first translates smart contracts to an intermediate language named SlithIR for further analysis. Then, Slither utilizes the semantic information with pre-defined vulnerable code patterns for vulnerability detection. Additionally, the ERC standard related information extracted by our approach can enhance the detection capabilities of Slither. For example, Slither's

“arbitrary-send-eth” detector does not currently recognize ERC721-defined operations, such as the `approve()` function, as send operations, causing it to fail in detecting the arbitrary-send vulnerability illustrated in Figure 11. Similarly, Slither lacks the ability to identify indirect exploitable operations, leading to missed detections like the arbitrary-send case shown in Figure 12. Our approach, however, extracts these semantic information, which can complement and improve Slither's existing vulnerability detection mechanisms. DefectChecker [35] is an efficient symbolic execution-based smart contract vulnerability detection tool. It uses stack events and features for pre-defined vulnerable pattern detection with less runtime consumption than existing symbolic execution-based tools. SoliDetector [8] is a syntax feature-based vulnerability detection tool. It first converts each smart contract to a knowledge graph by syntax analysis. Then, it utilizes W3C Web Ontology language [36] for vulnerable pattern description with SPARQL [37] for vulnerable pattern detection. SmartReco [38] is the first static analysis tool designed specifically for detecting Read-Only Reentrancy vulnerabilities in cross-DApp scenarios. Unlike other approaches, SmartReco leverages transaction replay to track and analyze the complex interactions between multiple DApps. This capability enables SmartReco to detect the Read-Only Reentrancy vulnerabilities arising from inter-DApp interactions, which cannot be identified by all existing approaches. NFTGuard [12] is a symbolic execution based vulnerability detection approach. It also utilize the ERC721 standard for hijackable call identification. After locating this API in the smart contract, the NFTGuard utilizes symbolic execution to extract the context information of this API. For the extracted context information, the NFTGuard checks if there is any state variable writing after this API calls for reentrancy vulnerability detection.

B. Deep Learning Based Based Reentrancy Vulnerability Detection

Meanwhile, with the rapid growth of AI technology, some deep learning-based approaches have been proposed. TMP [39] proposes to represent the smart contract code as a graph with the graph neural network for reentrancy vulnerability detection. CGE [40] proposes to combine the graph feature with designed expert patterns for smart contract vulnerability detection. It combines well-known security code patterns with code graphs as the input to train a vulnerability detection model. DeeSCVHunter [41] proposes to represent smart contract code as a token sequence with the LSTM model for reentrancy vulnerability detection. Peculiar [42] is a pre-training model-based vulnerability detection approach for smart contracts. It utilizes the DFG with pre-trained mode

GraphCodeBert [43] for vulnerability detection. Instead of training vulnerability detection models only by the smart contract code, Yuan et al. [44] propose combining multi-modality code for vulnerability detection. They propose to combine Solidity source code, assembly code, and intermediate representation as the model's input. Meanwhile, during vulnerability detection model training, they utilize a transformer encoder for multi-modality code feature learning.

VIII. CONCLUSION

This paper presents a static analysis-based framework for detecting reentrancy vulnerability in smart contracts. Our approach leverages static analysis with smart contract technical standards to identify potential entry and reentry points in smart contracts. For entry point identification, we detect hijackable external calls with delayed updated state variables, which are treated as taint sources. Then, in reentry point identification, we use data-and-control dependency relations along with exploitable operation identification to identify all potential reentry points containing direct or indirect exploitable operations. Finally, we employ static taint analysis to track the taint propagation and check if these taint sources can reach the direct or indirect exploitable operations within reentry points. Our approach significantly outperforms the baselines on our constructed dataset with ground truth. Additionally, it effectively detects reentrancy vulnerability within the dataset containing wild smart contracts publicly collected from Etherscan. In future work, we aim to enhance our approach to support vulnerability detection in smart contracts implemented using other programming languages.

REFERENCES

- [1] Z. Zheng et al., "An overview on smart contracts: Challenges, advances and platforms," *Future Gener. Comput. Syst.*, vol. 105, pp. 475–491, Apr. 2020.
- [2] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen, "Defining smart contract defects on Ethereum," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 327–345, Jan. 2022.
- [3] M. I. Mehar et al., "Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack," *J. Cases Inf. Technol.*, vol. 21, no. 1, pp. 19–32, 2019.
- [4] S. Badraddoja, R. Dantu, Y. He, K. Upadhayay, and M. Thompson, "Making smart contracts smarter," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, Vienna, Austria, May 2021, pp. 1–3.
- [5] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "Smartcheck: Static analysis of Ethereum smart contracts," in *Proc. 1st Int. Workshop Emerg. Trends Softw. Eng. Blockchain*, 2018, pp. 9–16.
- [6] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 67–82.
- [7] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB)*, May 2019, pp. 8–15.
- [8] P. Bose, D. Das, Y. Chen, Y. Feng, C. Kruegel, and G. Vigna, "SAILFISH: Vetting smart contract state-inconsistency bugs in seconds," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 161–178.
- [9] *Mythril*, MIT Press, Cambridge, MA, USA, 2025.
- [10] *Solhint is an Open-source Project to Provide a Linting Utility for Solidity Code*, MIT Press, Cambridge, MA, USA, 2025.
- [11] T. Hu, B. Li, Z. Pan, and C. Qian, "Detect defects of solidity smart contract based on the knowledge graph," *IEEE Trans. Rel.*, vol. 73, no. 1, pp. 1–17, Mar. 2024.
- [12] S. Yang, J. Chen, and Z. Zheng, "Definition and detection of defects in NFT smart contracts," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 373–384.
- [13] C. F. Torres, A. K. Iannillo, A. Gervais, and R. State, "Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts," in *Proc. IEEE Eur. Symp. Security Privacy (EuroSP)*, Sep. 2021, pp. 103–119.
- [14] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, "SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Nov. 2021, pp. 227–239.
- [15] C. Shou, S. Tan, and K. Sen, "ItFuzz: Snapshot-based fuzzer for smart contract," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 322–333.
- [16] M. Rodler et al., "EF/CF: High performance smart contract fuzzing for exploit generation," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy (EuroSP)*, Jul. 2023, pp. 449–471.
- [17] Y. Xue, M. Ma, Y. Lin, Y. Sui, J. Ye, and T. Peng, "Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2020, pp. 1029–1040.
- [18] J. Ye, M. Ma, Y. Lin, Y. Sui, and Y. Xue, "Clairvoyance: Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, Oct. 2020, pp. 274–275.
- [19] F. Vogelsteller and V. Buterin, "Erc-20 token standard," Ethereum Improvement Proposals, Tech. Rep. 20, Nov. 2015. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-20>
- [20] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze solidity smart contracts," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Melbourne, VIC, Australia, Dec. 2020, pp. 1349–1352.
- [21] Z. Zheng, N. Zhang, J. Su, Z. Zhong, M. Ye, and J. Chen, "Turn the rudder: A beacon of reentrancy detection for smart contracts on Ethereum," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA, May 2023, pp. 295–306.
- [22] W. Entriken. (2018). *Erc-721 Non-Fungible Token Standard*. EIPS/eip-721. Accessed: Sep. 10, 2021. [Online]. Available: <https://eips.ethereum.org>
- [23] J. Dafflon, J. Baylina, and T. Shababi. (2021). *Erc-777 Token Standard (2015)*. Ethereum Found. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-777>
- [24] *Eth: Currency for Our Digital Future*, Ethereum, Switzerland, 2025.
- [25] C. Dannen, *Introducing Ethereum and Solidity*, vol. 1. Cham, Switzerland: Springer, 2017.
- [26] R. A. C. Witek et al. (2021). *Erc1155: Multi Token Standard*. Ethereum Found. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1155>
- [27] M. Ortner and S. Eskandari. (Jul. 13, 2023). *Smart Contract Sanctuary*. [Online]. Available: <https://github.com/tintinweb/smart-contract-sanctuary>
- [28] *Mythx: Smart Contract Security Service for Ethereum*, Consensys, USA, 2020.
- [29] M. Arias, J. D. Fernández, M. A. Martínez-Prieto, and P. de la Fuente, "An empirical study of real-world SPARQL queries," 2011, *arXiv:1103.5043*.
- [30] H. Ritzdorf, Q. Higon, and P. Tsankov. *The Code Repository of Securify Version 1*. Accessed: Jan. 24, 2020. [Online]. Available: <https://github.com/eth-sri/securify>
- [31] Y. Sachinoglou, P. Tsankov, and C. Nuckols. *The Code Repository of Securify Version 2*. Accessed: Sep. 5, 2021. [Online]. Available: <https://github.com/eth-sri/securify2>
- [32] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Montreal, QC, Canada, Oct. 2020, pp. 530–541.
- [33] M. Kaleem, A. Mavridou, and A. Laszka, "Vyper: A security comparison with solidity based on common vulnerabilities," in *Proc. 2nd Conf. Blockchain Res. Appl. Innov. Netw. Services (BRAINS)*, Sep. 2020, pp. 107–111.
- [34] J. Avigad, L. Goldberg, D. Levit, Y. Seginer, and A. Titelman, "A verified algebraic representation of Cairo program execution," in *Proc. 11th ACM SIGPLAN Int. Conf. Certified Programs Proofs*, Jan. 2022, pp. 153–165.
- [35] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen, "DefectChecker: Automated smart contract defect detection by analyzing EVM bytecode," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2189–2207, Jul. 2022.
- [36] D. L. McGuinness and F. Van Harmelen, "Owl web ontology language overview," *W3C Recommendation*, vol. 10, no. 10, p. 2004, 2004.
- [37] B. DuCharme, *Learning SPARQL: Querying and Updating With SPARQL I.I.* Sebastopol, CA, USA: O'Reilly Media, 2013.

- [38] J. Zhang et al., "SmartReco: Detecting read-only reentrancy via fine-grained cross-DApp analysis," 2024, *arXiv:2409.18468*.
- [39] Y. Zhuang, Z. Liu, P. Qian, T. Wang, J. Li, and X. Zhang, "Smart contract vulnerability detection using graph neural networks," in *Proc. 29th Int. Conf. Int. Joint Conf. Artif. Intell. (IJCAI)*, Montreal, QC, Canada, Aug. 2021, pp. 3283–3290.
- [40] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
- [41] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Wu, "DeeSCVHunter: A deep learning-based framework for smart contract vulnerability detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Shenzhen, China, Jul. 2021, pp. 1–8.
- [42] H. Wu et al., "Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques," in *Proc. IEEE 32nd Int. Symp. Softw. Rel. Eng. (ISSRE)*, Hong Kong, Oct. 2021, pp. 378–389.
- [43] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [44] D. Yuan, X. Wang, Y. Li, and T. Zhang, "Optimizing smart contract vulnerability detection via multi-modality code and entropy embedding," *J. Syst. Softw.*, vol. 202, Aug. 2023, Art. no. 111699.



Jie Cai received the master's degree from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2016, and the Ph.D. degree from Yangzhou University, Yangzhou, China, in 2024. He was the Research Assistant at Macau University of Science and Technology from 2023 to 2024, supervised by Prof. Tao Zhang. He is currently a Lab Master with the School of Information Engineering, Yangzhou University. His research interests include blockchain security, software safety, and security.



Jiachi Chen (Member, IEEE) received the Ph.D. degree from the Faculty of Information Technology, Monash University, Australia. He is currently an Assistant Professor with the School of Software Engineering, Sun Yat-sen University. His research interests include blockchain, smart contracts, mining software repository, software security, and empirical study.



Tao Zhang (Senior Member, IEEE) received the B.S. degree in automation and the M.Eng. degree in software engineering from Northeastern University, China, and the Ph.D. degree from the Department of Computer Science and Engineering, University of Seoul. He is currently an Associate Professor with the School of Computer Science and Engineering, Macau University of Science and Technology (MUST), Macau, SAR. After that, he spent one year with The Hong Kong Polytechnic University as a Post-Doctoral Research Fellow. He has published more than 100 high-quality papers in renowned software engineering and security journals and conferences, such as ICSE, ESEC/FSE, ASE, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, TOSEM, EMSE, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING. His current research interests include AI for software engineering, software security, and mining software repositories. He is a Senior Member of ACM. He serves/served as the General or Program Chair for numerous academic conferences, including APSEC 2025, Internetwork 2024, SANER 2023, and DSA 2021. He also serves as a PC member for several top-tier SE conferences, including ICSE, FSE, ASE, and ISSTA. He is an Associate Editor of IEEE TRANSACTIONS ON SOFTWARE ENGINEERING (TSE) and the *Journal of Systems and Software* (JSS). He is also an Editorial Board Member of *Empirical Software Engineering* (EMSE) and *Science of Computer Programming* (SCP).



Xiapu Luo is currently a Professor with the Department of Computing, The Hong Kong Polytechnic University. His research focuses on blockchain and smart contracts security, mobile and IoT security, network security and privacy, and software engineering with papers published in top-tier security, software engineering, and networking conferences and journals. His research led to more than ten best/distinguished paper awards, including ACM CCS'24 Distinguished Paper Award, four ACM SIGSOFT Distinguished Paper Awards, Best DeFi Papers Award in 2023, Best Paper Award in INFOCOM'18, and Best Research Paper Award in ISSRE'16; and several awards from the industry. He received the BOCHK Science and Technology Innovation Prize (FinTech) 2023 for his contribution to blockchain security. He is an ACM Distinguished Member for his research in safeguarding blockchain and smart contracts along with Android and its applications.



Xiaobing Sun received the B.S. degree in computer science and technology from Jiangsu University of Science and Technology, Zhenjiang, China, in 2007, and the Ph.D. degree from the School of Computer Science and Engineering, Southeast University, Nanjing, China, in 2012. He is currently a Professor with the School of Information Engineering, Yangzhou University, Yangzhou, China. He has been authorized more than 20 patents. He has published more than 80 articles in refereed international journals. His research interests include software maintenance and evolution, software repository mining, and intelligence analysis.



Bin Li received the B.S. degree in computer software from Fudan University, Shanghai, China, in 1986, and the M.S. and Ph.D. degrees in computer application technology from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 1993 and 2001, respectively. He is currently a Professor with Yangzhou University, Yangzhou, China. He has published more than 100 journal articles and conference papers. His main research interests include knowledge graph, software data mining, and multiagent systems.